Smart RC Book

Minimum Viable Product — Technical Specification

Version 1.0 | IoT-based Government Vehicle RC Verification System

1. Executive Summary

Smart RC Book is an IoT system that digitises the roadside vehicle Registration Certificate (RC) verification process used by traffic police and RTO officers in India. The officer carries a compact handheld device — an ESP8266 microcontroller with an RFID reader and an OLED display — that reads an RFID tag embedded in a vehicle's windshield sticker. The device queries a cloud-hosted PHP/MySQL backend and displays the requested vehicle details on the OLED within 1–2 seconds, eliminating the need to carry physical RC booklets or use bulky desktop terminals.

The MVP covers the complete end-to-end flow: RFID tag provisioning, cloud API, admin dashboard for data management, and hardware firmware. All components are production-ready and have been reviewed and hardened against common security issues.

2. Problem Statement

Current RC verification in the field suffers from three core problems:

- Physical RC booklets can be forged, damaged, or left at home. Officers have no reliable way to verify authenticity on the spot.
- Mobile apps require smartphones with internet connectivity and are slow to navigate during traffic checks.
- Centralised terminal desks at RTOs are not portable — officers on highway patrol have no access.

Smart RC Book solves all three: the RFID tag is tied to a database record (not the paper document), the handheld device is purpose-built and fast, and the cloud backend can be accessed from anywhere with a data connection.

3. System Architecture

The system has three layers:

Layer	Components
Hardware (Field Device)	ESP8266 NodeMCU, RC522 RFID reader, SH1106 1.3" OLED, 3 push buttons
Backend (Cloud Server)	PHP 8 REST API, MySQL (RC_DATA + admin_login databases), Apache + .htaccess
Admin Interface (Browser)	PHP dashboard — vehicle records, search, edit, compliance overview

3.1 Data Flow

1. Officer taps vehicle RFID tag with the handheld reader.
2. ESP8266 extracts the 8-hex-character UID from the card.
3. Device sends HTTPS GET request to the cloud API with the UID, selected field group, and API key.
4. Server looks up the UID in RC_DATA, joins across 6 tables, and returns a compact JSON payload.
5. ESP8266 parses the JSON and renders up to 5 key-value rows on the OLED.
6. Officer reads the result within 1–2 seconds of the tap.

4. Hardware Specification

4.1 Components

Component	Role
ESP8266 NodeMCU / Wemos D1 Mini	Main microcontroller — WiFi, HTTP, logic
MFRC522 RC522 RFID Reader	Reads MIFARE Classic card UID via SPI
SH1106 1.3" OLED (128×64)	Displays menus and RC data (I2C)
BTN_NAV — pushbutton on D0	Navigate menu items (next)
BTN_SEL — pushbutton on A0	Confirm selection
BTN_BACK — pushbutton on D4	Return to previous screen
10kΩ resistors (×2)	External pull-ups for BTN_NAV and BTN_SEL

4.2 Pin Assignment

ESP8266 Pin	Connected to
D8 (GPIO15)	RC522 SDA (SS)
D5 (GPIO14)	RC522 SCK
D7 (GPIO13)	RC522 MOSI
D6 (GPIO12)	RC522 MISO
D3 (GPIO0)	RC522 RST
D2 (GPIO4)	OLED SDA
D1 (GPIO5)	OLED SCL
D0 (GPIO16)	BTN_NAV (10kΩ pull-up to 3.3V)
A0	BTN_SEL (10kΩ pull-up to 3.3V)
D4 (GPIO2)	BTN_BACK (internal pull-up)

> All peripherals operate at 3.3V. Do NOT connect RC522 or OLED to 5V — permanent damage will result. D4 (GPIO2) and D3 (GPIO0) must be HIGH at boot.

5. Firmware

The firmware is written in Arduino C++ targeting the ESP8266 board package. Key design decisions:

5.1 State Machine

State	Description
STATE_WIFI	Boot — connecting to WiFi with animated dots
STATE_MENU	Field group selection menu (8 options)
STATE_READY	Waiting for RFID tap — shows group + scan count
STATE_FETCHING	HTTPS request in flight — spinner animation
STATE_RESULT	RC data displayed — scrollable, 5 rows visible
STATE_NOT_FOUND	Tag UID not in database — shows UID for reporting
STATE_ERROR	HTTP or parse failure — shows error code
STATE_WIFI_RECONNECTING	Manual WiFi reconnect initiated from menu

5.2 Menu Groups

The officer selects which data to request before tapping the card. This minimises the API response size and keeps the OLED readable.

Menu Option	Fields Requested
Vehicle Info	vehicle_no, model, fuel_type, color, class
Owner Details	owner_name, father_name
Insurance	insurance expiry status
Fitness Cert	fitness expiry status
Road Tax	tax paid_upto
Pollution PUC	pollution expiry + PUCC number
Compliance	insurance, fitness, tax, pollution (all 4)
Full Details	owner, vehicle no, model, insurance, fitness
Reconnect WiFi	Special action — triggers WiFi reconnection

5.3 Key Features

- Result cache — same card within 60 seconds returns instantly without a network request. Badge [C] shown on result.
- Cache is group-aware — changing the field group and scanning the same card always fetches fresh data.
- OLED sleep — display turns off after 3 minutes of inactivity to prevent burn-in.
- WiFi auto-reconnect — background retry every 10 seconds when connection is lost.
- Server health check on boot — confirms API is reachable before showing the menu.
- Scan counter — session total shown on the ready screen.
- HTTPS with BearSSL — all API traffic is encrypted (setInsecure mode by default; fingerprint pinning available).

5.4 Bug Fixes Applied (v3.1)

- Button event queue (circular buffer) — simultaneous presses no longer lost.
- All buttons fire on press — consistent feel across NAV, SEL, BACK.
- Forward declarations — safe compilation order for all functions.
- Cache key includes menuIndex — prevents stale data when changing group.
- drawNotFound shows UID on row 2 — officer can read and report unknown tags.
- WiFi reconnect checks wl_status_t — avoids interrupting in-progress connection attempts.
- Non-blocking RFID poll via millis() — delay(50) removed from main loop.
- char[] buffers throughout — eliminates String heap fragmentation on long sessions.

6. Backend — API & Database

6.1 Database Schema

Two databases are used:

Database	Purpose
admin_login	Admin authentication — stores email, bcrypt password, rotating session token
RC_DATA	Vehicle records — 6 tables joined on rc_no

RC_DATA tables:

Table	Key Fields
rc	rc_no (PK), vehicle_no, model, manufacturer, fuel_type, color, engine_no, chassis_no, rto, state, registration_valid_upto, rfid_tag
owners	rc_no (FK), name, father_name, address
insurance	rc_no (FK), policy_no, provider, valid_upto
fitness	rc_no (FK), valid_upto
tax	rc_no (FK), paid_upto
pollution	rc_no (FK), pucc_no, valid_upto

6.2 RFID API Endpoint

The hardware calls a single endpoint:

```
GET
https://example.com/smartrc/api/rfid?tag=A1B2C3D4&fields=insurance,fitness&key=API_KEY&compact=1
```

Parameter	Description
tag	8–16 hex char RFID UID from the card
fields	Comma-separated list of fields to return
key	API key — server rejects requests without it
compact	1 = short values (OK/EXP/SOON); 0 = full dates

Response (found):

```
{ "found": true, "rc_no": "MH12AB1234", "data": { "ins": "OK", "fit": "SOON" } }
```

Response (not found):

```
{ "found": false, "message": "Tag not registered" }
```

6.3 Auth Flow

1. Officer/admin visits /smartrc/login in a browser.
2. POST credentials → server verifies bcrypt hash → returns session token.
3. JS uses hidden form POST (token never appears in URL or browser history).
4. Dashboard barrier validates token, sets 3 cookies (adminToken, isLoggedIn, adminName) with 8-hour TTL.
5. Token rotates on every login and password change.
6. Logout clears all 3 cookies.

6.4 Security Measures

- All passwords stored as bcrypt (cost 12).
- Prepared statements (mysqli_stmt) used for all database writes.
- SQL errors logged server-side — never shown to users.
- Internal PHP include files blocked from direct HTTP access via .htaccess.
- .sql, .bat, .env, .log files blocked at server level.
- HTTPS enforced via 301 redirect in .htaccess.
- HSTS header set (1 year, includeSubDomains).
- CORS limited to rfid.php only — dashboard routes have no CORS headers.
- X-Frame-Options, X-Content-Type-Options, Referrer-Policy headers set.

7. Admin Dashboard

The dashboard is a PHP/HTML web application accessible at https://example.com/smartrc/dashboard.

Page	Function
Login (/login)	Secure login form — token-based auth
Dashboard (/dashboard)	Stats tiles: total vehicles, expired insurance, fitness, pollution. Recent vehicles table with search.
Vehicle (/vehicle?rc=...)	View and edit all RC fields across all 6 tables. RFID tag is locked (read-only).
Change Password	Validates current password, enforces complexity, rotates session token.
Logout	Clears all session cookies instantly.

Dashboard JS features:

- Table rows are clickable — click anywhere on a row to open the vehicle page.
- Search bar has a clear (×) button and / keyboard shortcut to focus.

- Success alerts auto-dismiss after 4 seconds.
- Session expiry warning appears 5 minutes before the 8-hour cookie expires.

8. RFID Card Provisioning

Each vehicle is issued a MIFARE Classic RFID card. The provisioning process uses two helper sketches:

8.1 rfid_writer.ino

7. Set DATA_TO_WRITE to the RC number in the sketch (e.g. "MH12AB1234").
8. Flash to the ESP8266 and open Serial Monitor at 115200 baud.
9. Tap a blank MIFARE Classic card — the RC number is written to Block 1.
10. Serial Monitor prints the card UID (e.g. A1B2C3D4).
11. Run the SQL UPDATE to register the UID in the database:

```
UPDATE `rc` SET `rfid_tag` = 'A1B2C3D4' WHERE `rc_no` = 'MH12AB1234';
```

8.2 rfid_reader.ino

Reads any MIFARE card and prints UID, card type, and block 0–7 contents to Serial Monitor. Used to verify written cards and retrieve UIDs from existing cards.

> WARNING: Never write to Block 0 (manufacturer data — read-only) or trailer blocks 3, 7, 11, 15... (sector keys). Writing wrong data to a trailer block permanently locks the sector.

9. Deployment

9.1 Server Requirements

Requirement	Minimum
Web server	Apache 2.4+ with mod_rewrite
PHP version	8.0 or higher
MySQL / MariaDB	5.7+ (8.0 recommended for CHECK constraint support)
SSL certificate	Required — Let's Encrypt (free, auto-renew via Certbot)
Hosting path	/public_html/smartrc/ (shared hosting) or /var/www/html/smartrc/

9.2 Setup Steps

12. Upload the smartrc/ folder to the server's public directory.
13. Import admin_login_setup.sql and rc_schema.sql in phpMyAdmin.
14. Edit auth/api/adminDbConnection.php — set DB host, username, password.
15. Edit api/rfid.php — set API_KEY to a strong random string.
16. Edit .htaccess — confirm RewriteBase matches your path (/smartrc/).
17. Enable mod_rewrite and set AllowOverride All in Apache config.

18. Verify HTTPS is working — then enable HSTS header in .htaccess.
19. Flash smartrc.ino to ESP8266 with correct WIFI_SSID, WIFI_PASSWORD, SERVER_URL, API_KEY.
20. Log in at https://example.com/smartrc/login — default credentials: admin@smartrc.local / Admin@1234.
21. Change password immediately after first login.

10. Known Limitations & Upgrade Path

Limitation	Upgrade Path
ESP8266 RAM (~15-20KB free)	Upgrade to ESP32 (520KB RAM, dual core, hardware TLS)
http.getString() uses String heap	Stream deserialization with ArduinoJson (eliminates last heap allocation)
setInsecure() HTTPS mode	Enable SSL fingerprint pinning (update every cert renewal)
Single admin account	Add multi-user support with roles (officer/admin)
No offline mode	Local SQLite cache on device for areas with poor connectivity
Manual RFID provisioning	Bulk provisioning tool — CSV upload maps UIDs to RC numbers
No audit log	Add last_queried timestamp to rc table for access tracking

11. File Structure

File	Purpose
index.php	Root redirect
responsegenerator.php	Shared JSON response helpers, CORS
.htaccess	URL routing, HTTPS redirect, security headers, file blocking
auth/api/adminDbConnection.php	MySQLi connection to admin_login database
auth/api/getuserdata.php	generateRandomString(), getAdminByToken()
auth/login/index.php	Login page HTML
auth/login/login.php	POST endpoint — validates credentials, returns token
admin/dashboard/index.php	Auth barrier — validates token, sets session cookies
admin/dashboard/home/index.php	Dashboard — stats, search, vehicle table
admin/dashboard/home/vehicle.php	View and edit all RC fields
admin/dashboard/home/change_password.php	Password change with token rotation
admin/dashboard/home/logout.php	Clears all cookies
admin/dashboard/home/includes/db_rc.php	MySQLi helpers for RC_DATA
admin/dashboard/home/includes/header.php	Navigation header
api/rfid.php	ESP8266 RFID lookup endpoint
esp8266/smartrc.ino	Main firmware v3.1
esp8266/rfid_writer/rfid_writer.ino	Writes RC number to MIFARE card
esp8266/rfid_reader/rfid_reader.ino	Reads card UID and blocks to Serial Monitor
esp8266/rfid_oled_test/rfid_oled_test.ino	RFID + OLED hardware test sketch
admin_login_setup.sql	Creates admin_login DB and default admin account

12. Glossary

Term	Definition
RC	Registration Certificate — government document proving a vehicle is registered
RTO	Regional Transport Office — the authority that issues RC documents in India
RFID	Radio Frequency Identification — contactless wireless tag reading technology

MIFARE Classic	Common RFID card standard (ISO 14443A) used in access cards and transit passes
UID	Unique Identifier — factory-programmed 4 or 7-byte hardware serial on every MIFARE card
PUC / PUCC	Pollution Under Control Certificate — mandatory annual emission test certificate
ESP8266	Low-cost WiFi-enabled microcontroller by Espressif, used as the device brain
SPI	Serial Peripheral Interface — 4-wire synchronous serial bus used by the RC522
I2C	Two-wire serial bus used by the SSD1306/SH1106 OLED display
BearSSL	Lightweight TLS library built into the ESP8266 Arduino board package
bcrypt	Password hashing algorithm — computationally expensive, resistant to brute force
HSTS	HTTP Strict Transport Security — forces browsers to use HTTPS only

Smart RC Book — MVP v1.0 — All rights reserved